# ■ ¡VALIDACIÓN O MUERTE!

**Una comedia sobre Spring Validator, Jakarta y las validaciones personalizadas**

**5 integrantes · 8–10 minutos · Mucho humor + teoría**

**Personajes:** 1) Doctor Spring · 2) Programador Junior · 3) Jakarta Validator · 4) Usuario/Cliente · 5) Validador Personalizado

**Utilería opcional:** bata o gafas para el Doctor, una libreta para Jakarta, laptop/celular para el Usuario y un cartel que diga “ERROR 400”.

## ■ ESCENA 1 — ¡EMERGENCIA EN PRODUCCIÓN!

**USUARIO:** ¡DOCTOR SPRING! ¡DOCTOR SPRING! ¡CORRA, CORRA!

**DOCTOR SPRING:** ¿Qué pasó? ¿Se cayó el servidor?

**USUARIO:** ¡No!

**DOCTOR SPRING:** ¿Se borró la base de datos?

**USUARIO:** ¡No!

**DOCTOR SPRING:** ¿El programador hizo un *DELETE* sin *WHERE*?

**PROGRAMADOR JUNIOR:** ¡¿Por qué todos me miran?! ■

**USUARIO:** ¡Mi formulario acepta cualquier cosa! Puse nombre: **12345**, correo: **hola** y edad: **-50**.

**DOCTOR SPRING:** ¿Edad -50? ¡Ese usuario todavía no ha nacido! ■

**PROGRAMADOR JUNIOR:** Doctor, yo pensé que Java hacía magia.

**DOCTOR SPRING:** Java hace muchas cosas, pero tampoco puede arreglar tus malas decisiones.

**DOCTOR SPRING:** ¡Necesitamos validación!

## ■ ESCENA 2 — APARECE EL GUARDIA: JAKARTA VALIDATOR

**[Entra JAKARTA VALIDATOR con una libreta y cara seria.]**

**JAKARTA VALIDATOR:** ¡ALTO! ¡Nadie entra sin validación!

**USUARIO:** ¿Quién eres tú?

**JAKARTA VALIDATOR:** Soy Jakarta Validator. Mi trabajo es revisar que los datos cumplan las reglas.

**PROGRAMADOR JUNIOR:** ¿Y si no cumplen?

**JAKARTA VALIDATOR:** ¡RECHAZADO!

**USUARIO:** ¿Así de rápido?

**JAKARTA VALIDATOR:** Sí. Yo no negocio con datos sospechosos.

**DOCTOR SPRING:** La validación sirve para comprobar que los datos recibidos sean correctos antes de continuar con el procesamiento.

**PROGRAMADOR JUNIOR:** O sea, eres como el portero de una discoteca.

**JAKARTA VALIDATOR:** Exactamente. Si no cumples los requisitos, no pasas.

**USUARIO:** ¿Y si conozco al dueño?

**JAKARTA VALIDATOR:** ¡Ni con vara! ■

## ■ ESCENA 3 — EL ATAQUE DE LAS ANOTACIONES

**DOCTOR SPRING:** Para establecer reglas podemos usar anotaciones de Jakarta Validation.

**JAKARTA VALIDATOR:** Presento a mi equipo: **@NotNull, @NotBlank, @Size, @Email, @Min, @Max y @Pattern**.

**PROGRAMADOR JUNIOR:** ¿Todas trabajan para ti?

**JAKARTA VALIDATOR:** Sí. Somos el equipo que evita que entren datos locos.

**DOCTOR SPRING:** Por ejemplo, **@NotNull** indica que un valor no puede ser null.

**USUARIO:** Entonces si dejo el campo sin valor...

**JAKARTA VALIDATOR:** ¡RECHAZADO!

**DOCTOR SPRING:** **@NotBlank** evita que un texto esté vacío o tenga solo espacios.

**USUARIO:** ¿Y si pongo “ ”?

**JAKARTA VALIDATOR:** ¡Eso no es un nombre, eso es espacio desperdiciado! ■

**PROGRAMADOR JUNIOR:** ¿Y **@Size**?

**DOCTOR SPRING:** Controla la longitud. Por ejemplo: **@Size(min = 8, max = 20)**.

**USUARIO:** Entonces mi contraseña “123”...

**JAKARTA VALIDATOR:** ¡NI LO INTENTES!

**DOCTOR SPRING:** Y **@Email** comprueba que el correo tenga un formato válido.

**USUARIO:** ¿“hola”?

**JAKARTA VALIDATOR:** ¡Eso no es un correo, eso es un saludo!

## ■ ESCENA 4 — EL PROBLEMA: “QUIERO MI PROPIA REGLA”

**PROGRAMADOR JUNIOR:** Doctor, tengo un problema. Las anotaciones existentes no cubren una regla que necesito.

**DOCTOR SPRING:** Entonces crearemos una **validación personalizada**.

**USUARIO:** ¿Como cuando uno inventa sus propias reglas?

**DOCTOR SPRING:** Exactamente, pero con código y no con imaginación. ■

**[Entra VALIDADOR PERSONALIZADO con lentes o una capa.]**

**VALIDADOR PERSONALIZADO:** ¡Me llamaron! Yo puedo crear una regla especial.

**PROGRAMADOR JUNIOR:** ¿Qué puedes validar?

**VALIDADOR PERSONALIZADO:** Por ejemplo, un código de empleado que obligatoriamente tenga el formato **EMP-1234**.

**USUARIO:** ¿Y si pongo EMP-12?

**VALIDADOR PERSONALIZADO:** ¡RECHAZADO!

**USUARIO:** ¿EMP-12345?

**VALIDADOR PERSONALIZADO:** ¡RECHAZADO!

**USUARIO:** ¿EMP-1234?

**VALIDADOR PERSONALIZADO:** ¡APROBADO! Puede continuar, ciudadano. ■

**DOCTOR SPRING:** Una validación personalizada normalmente se construye creando una **anotación propia** y un **validador** que contiene la lógica para comprobarla.

## ■ ESCENA 5 — JAKARTA VS. SPRING: ¡NO SON LO MISMO!

**USUARIO:** Tengo una duda. ¿Jakarta Validation y Spring Framework son lo mismo?

**JAKARTA VALIDATOR:** ¡NO!

**DOCTOR SPRING:** ¡NO!

**PROGRAMADOR JUNIOR:** ¡Entonces me van a reprobar! ■

**DOCTOR SPRING:** Tranquilo. Jakarta Validation proporciona la especificación y las reglas de validación, incluyendo las anotaciones.

**JAKARTA VALIDATOR:** Yo pongo las reglas.

**DOCTOR SPRING:** Y Spring Framework integra esas validaciones dentro de la aplicación.

**VALIDADOR PERSONALIZADO:** Y yo aparezco cuando necesitamos reglas hechas a medida.

**USUARIO:** Entonces son un equipo.

**DOCTOR SPRING:** Exactamente.

**PROGRAMADOR JUNIOR:** Como los Avengers.

**JAKARTA VALIDATOR:** Solo que nosotros luchamos contra los datos incorrectos.

**USUARIO:** ¡Y contra el programador que pone -50 años! ■

## ■ ESCENA 6 — ¿CÓMO SE APLICA EN SPRING?

**DOCTOR SPRING:** Veamos un ejemplo práctico.

**PROGRAMADOR JUNIOR:** Tenemos una clase **Usuario**:

```
public class Usuario {
@NotBlank
private String nombre;

@Email
private String correo;

@Size(min = 8)
private String contraseña;
}
```

**USUARIO:** Entonces las reglas están colocadas sobre los atributos.

**DOCTOR SPRING:** Correcto. Y en el controlador podemos utilizar **@Valid** para indicar que queremos validar el objeto recibido.

```
@PostMapping("/usuarios")
public ResponseEntity<?> registrar(
@Valid @RequestBody Usuario usuario) {
return ResponseEntity.ok("Usuario registrado");
}
```

**JAKARTA VALIDATOR:** Cuando llegan los datos, ¡yo reviso las reglas!

**VALIDADOR PERSONALIZADO:** Y si hay una regla especial, ¡yo la compruebo!

**DOCTOR SPRING:** Si hay errores, la aplicación puede devolver información indicando qué datos no cumplen las validaciones.

**PROGRAMADOR JUNIOR:** ¡Entonces el error se detecta antes de que llegue a hacer un desastre!

**DOCTOR SPRING:** Exactamente. Eso es lo que queremos.

## ■ ESCENA 7 — EL EXAMEN SORPRESA

**DOCTOR SPRING:** Ahora vamos a hacer un examen rápido. Si se equivocan, ¡Jakarta los valida! ■

**DOCTOR SPRING:** ¿Qué anotación usamos para un correo?

**TODOS:** ¡@Email!

**DOCTOR SPRING:** ¿Para evitar un texto vacío o solo espacios?

**TODOS:** ¡@NotBlank!

**DOCTOR SPRING:** ¿Para controlar longitud?

**TODOS:** ¡@Size!

**DOCTOR SPRING:** ¿Para establecer un mínimo o máximo numérico?

**TODOS:** ¡@Min y @Max!

**DOCTOR SPRING:** ¿Y si necesitamos una regla que no existe?

**TODOS:** ¡VALIDACIÓN PERSONALIZADA!

**DOCTOR SPRING:** ¿Y qué usamos en Spring para indicar que queremos validar el objeto recibido?

**TODOS:** ¡@Valid!

## ■ ESCENA 8 — EL FINAL MÁS IMPORTANTE

**USUARIO:** Ya aprendí. Voy a registrarme correctamente.

**USUARIO:** Nombre: Alexis.

**JAKARTA VALIDATOR:** ¡Correcto!

**USUARIO:** Correo: alexis@gmail.com.

**JAKARTA VALIDATOR:** ¡Correcto!

**USUARIO:** Contraseña: 123.

**JAKARTA VALIDATOR:** ¡RECHAZADO!

**USUARIO:** ¡Otra vez! ■

**DOCTOR SPRING:** ¡Mínimo 8 caracteres!

**USUARIO:** Está bien... “spring123”.

**JAKARTA VALIDATOR:** ¡APROBADO!

**PROGRAMADOR JUNIOR:** ¡Por fin! ¡Producción salvada!

**VALIDADOR PERSONALIZADO:** Recuerden: si una regla no existe, ¡podemos crearla!

**JAKARTA VALIDATOR:** Las anotaciones permiten definir reglas de validación.

**DOCTOR SPRING:** Spring Framework integra estas validaciones en nuestras aplicaciones.

**USUARIO:** Y @**Valid** permite activar la validación del objeto recibido.

**PROGRAMADOR JUNIOR:** Entonces una aplicación sin validación es como una puerta sin seguridad.

**JAKARTA VALIDATOR:** ¡Entra cualquiera!

**USUARIO:** ¡Hasta el de -50 años! ■

**TODOS:** ¡VALIDAR ANTES DE PROCESAR! ¡FIN! ■

# ■ CHULETA PARA LA EXPOSICIÓN

• **Spring Validator:** mecanismo utilizado para comprobar que los datos recibidos cumplen determinadas reglas.

• **Anotaciones:** permiten declarar reglas de validación de forma sencilla. Ejemplos: @NotNull, @NotBlank, @Size, @Email, @Min, @Max y @Pattern.

• **Jakarta Validation:** proporciona la especificación y las anotaciones de validación.

• **Spring Framework:** integra estas validaciones en aplicaciones Spring.

• **Validación personalizada:** permite crear una anotación propia y un validador con lógica específica.

• **Aplicación:** las reglas pueden colocarse en clases de datos/DTOs y activarse en controladores mediante @Valid.

> ■ **Consejo para que dé risa en clase:** hagan pausas, exageren cuando Jakarta diga “¡RECHAZADO!”, y que el Programador Junior siempre tenga una excusa absurda. No lean todo mirando el papel: actúen las entradas y salidas.